

RELATO

Envcheck desenvolvimento orientado por especificação com Spec Kit

Projeto. CLI em Rust para validar arquivos `.env` contra `.env.example` ou `.env.schema` TOML tipado. A execução é determinística, local e somente leitura; valores reais do target não aparecem nos diagnósticos. O discovery procura primeiro `.env.schema` e depois `.env.example` ao lado do arquivo alvo.

Fluxo utilizado no Spec Kit

- 01 Instalação / inicialização**
preparação do repositório com `.specify/` e skills do agente em `.agents/skills/`.
- 02 \$speckit-constitution**
correção, estabilidade do CLI, separação de responsabilidades, testes, segurança, dependências mínimas, portabilidade, operação local/in-memory e quality gates.
- 03 \$speckit-specify**
requisitos funcionais e critérios mensuráveis para example, schema, discovery, diagnósticos e exit codes.
- 04 \$speckit-clarify**
resolução de ambiguidades sobre identificadores, comentários, vazio/missing, números, regex, allowed, duplicatas e redaction.
- 05 \$speckit-plan**
Rust stable, clap, serde/toml, regex e dotenvx-primitives::scan; parser, schema, domínio e output desacoplados.
- 06 \$speckit-checklist**
revisão da qualidade dos requisitos depois do plan; ao final, 16/16 e 48/48 itens revisados.
- 07 \$speckit-tasks**
decomposição do trabalho em tarefas dependentes com abordagem incremental e TDD.
- 08 \$speckit-analyze**
análise cruzada entre spec, plan, tasks, constituição e código antes da implementação para localizar inconsistências.
- 09 \$speckit-implement**
ciclos RED/GREEN: testes primeiro, implementação mínima depois e reexecução dos gates.
- 10 \$speckit-converge**
geração de T057/T058 para corrigir gate de release e precisão numérica acima de 2^{53} , seguida de nova implementação.

Uso da aplicação

```
# .env.example explícito
envcheck .env --example .env.example
```

```
# schema tipado explícito
envcheck .env --schema .env.schema
```

```
# discovery ao lado do target
envcheck config/.env
```

No discovery, `.env.schema` tem precedência sobre `.env.example`. Exit codes públicos: 0 válido/warnings; 1 erro de validação; 2 uso incorreto; 3 input/discovery/dotenv; 4 schema inválido. A pasta `examples/` cobre casos simples, complexos, apenas `example`, apenas `schema`, ambos, `override` explícito, `schema` inválido e ausência de definição.

Estrutura resultante

src/	
cli.rs	argumentos do CLI
env/	parsing dotenv
schema/	modelo + TOML
validation/	regras e diagnósticos
output/	stdout/stderr
tests/cli.rs	testes do binário
specs/001-env-validation/	spec.md, plan.md, tasks.md
	quickstart.md, contracts/, checklists/
examples/	9 cenários executáveis
.github/workflows/	ci.yml, release.yml

Experiência com o processo

Na prática, o fluxo funcionou melhor quando os artefatos foram tratados como parte do trabalho, e não só como documentação. O Clarify ajudou a fechar dúvidas ainda na especificação, antes de elas virarem decisões de implementação. Depois do Plan, o Checklist serviu para conferir se os requisitos e o desenho continuavam coerentes antes de quebrar o trabalho em tarefas. O Analyze, usado antes do Implement, funcionou como uma revisão cruzada entre spec, plan, tasks e constituição e mostrou pontos que ainda estavam soltos. Depois da implementação, o Converge transformou os gaps restantes em tarefas objetivas, que voltaram para o mesmo ciclo de código e testes. Esse vai e volta deixou as etapas mais consistentes entre si e evitou que decisões importantes ficassem só no código ou fossem resolvidas tarde demais.